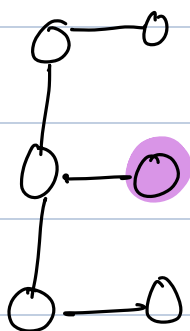


- 1, Bellman Ford Algorithm
- 2, Travelling Salesman problem.

SSSP Algo

BFS

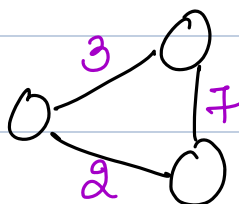
(Un weighted graph)



Dijk

[weighted graph]

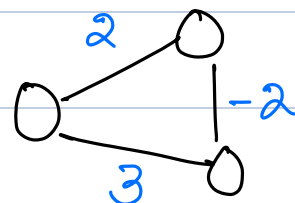
+ve



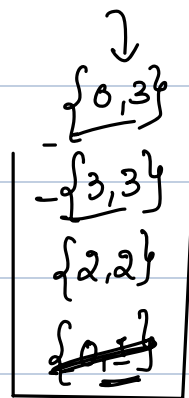
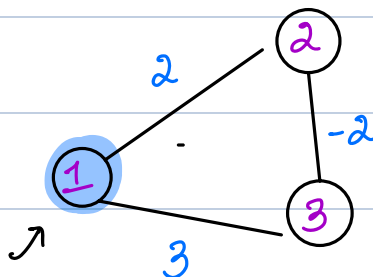
Bellman-ford

[weighted graph]

-ve

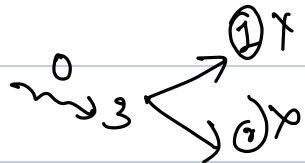
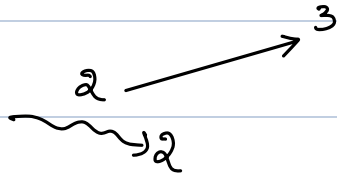
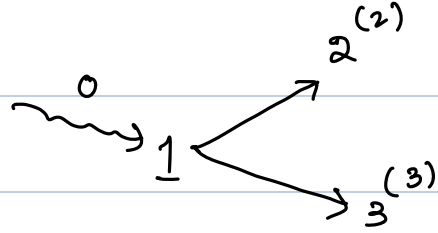


Eg:-



1	2	3
0	∞	∞

1	2	3
∞	∞	0



1) Bellman ford :

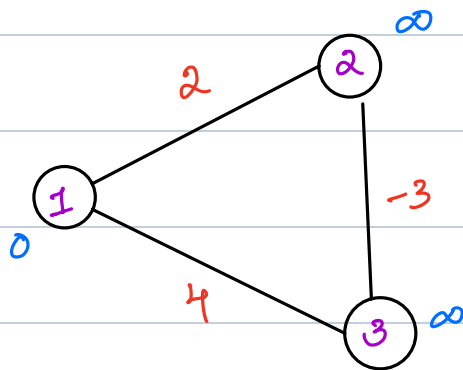
=> Bellman ford Algorithm Steps.

1, Init $\text{dist}[\text{vertices}] = \text{INT_MAX}$,

$\text{dist}[\text{src}] = 0$

2, Relax all edges $v-1$ times [Most Important]

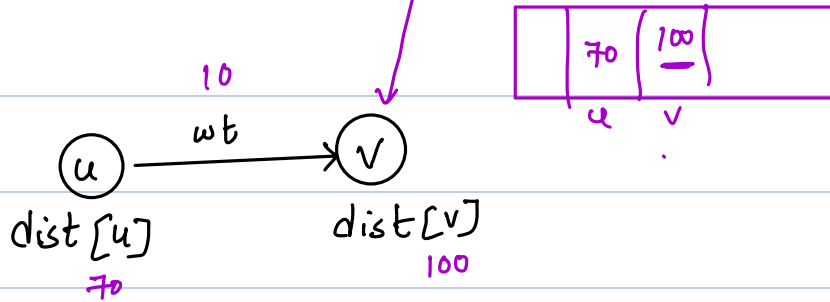
3, Negative weight Cycle Detection.



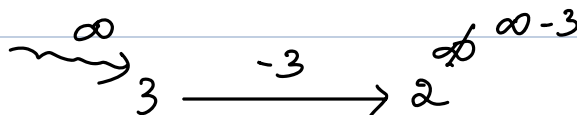
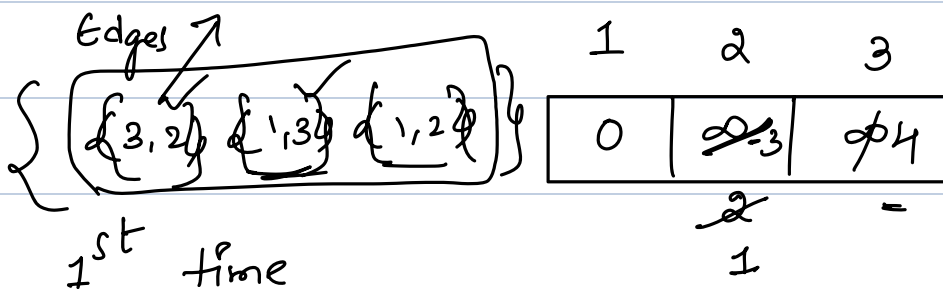
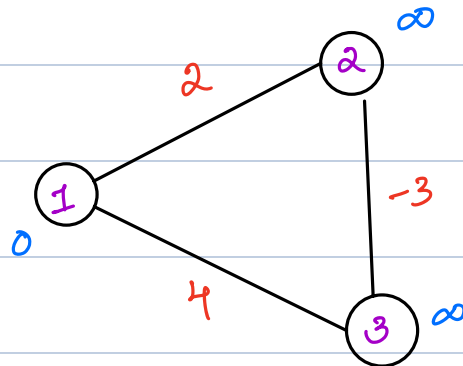
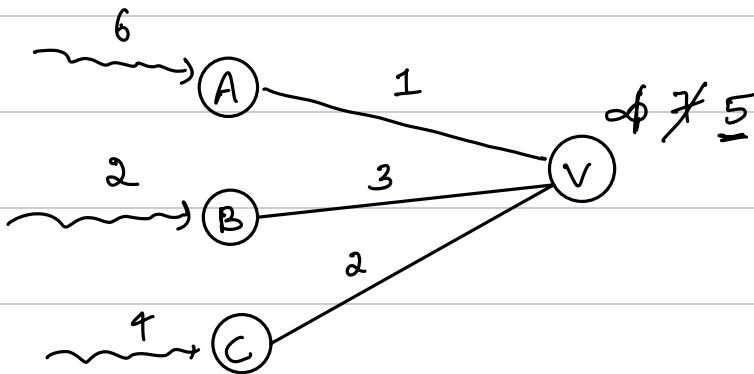
Edges

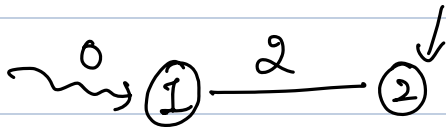
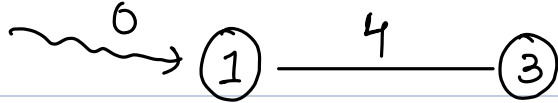
$(3,2)$ $(1,3)$ $(1,2)$

Relaxation.

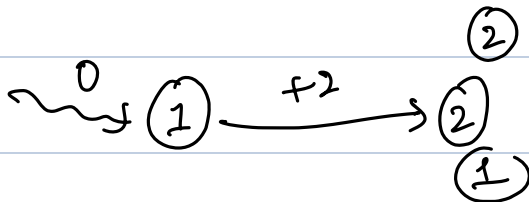
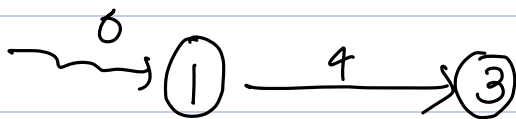
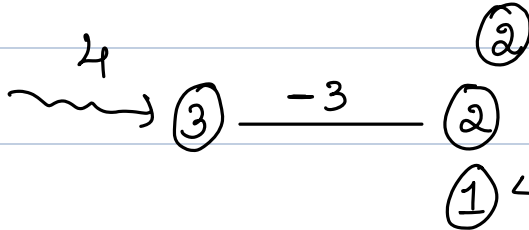


if ($dist[v] > dist[u] + wt_{uv}$)
 $dist[v] = dist[u] + wt_{uv}$



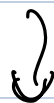


2nd time



Worst Case

Code:-



Vector <vector <int>> edges.

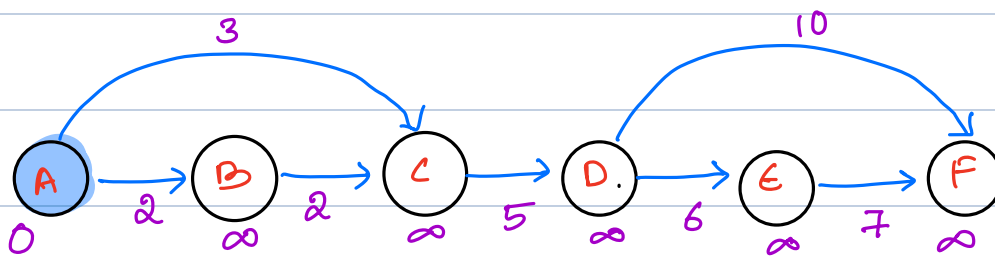
Vector <int> dist (v+1, ∞);

dist[src] = 0;

```

for (int i=0 ; i<V-1 ; i++) {
    for (auto edg : edges) {
        int u = edg[0];
        int v = edg[1];
        int wt = edg[2];
        if (dist[u] == ∞ and dist[v] >
            dist[u] + wt) {
            dist[v] = dist[u] + wt;
        }
    }
}

```



Edge list

A-B, B-C, A-C, C-D, D-E, E-F, D-F

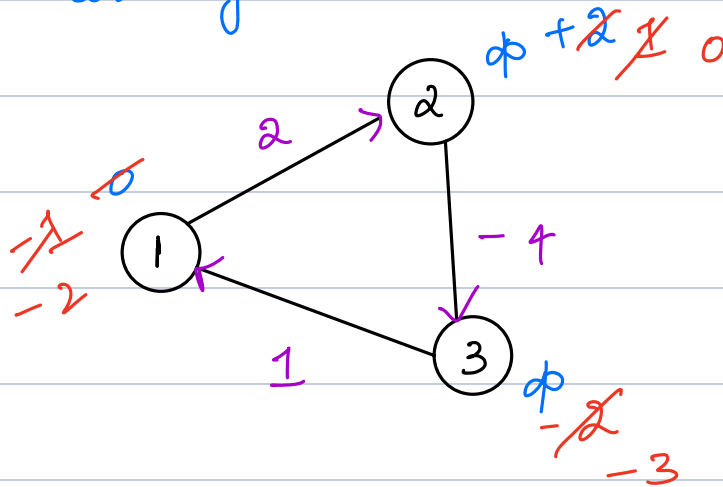
	A	B	C	D	E	F
1 st	0	∅	∅	∅	∅	∅
		2	4	8	14	24

3 10

$\checkmark$ D-F, $\checkmark$ E-F, $\checkmark$ D-E, $\checkmark$ C-D, $\checkmark$ A-C, $\checkmark$ B-C, A-B

A	B	C	D	E	F
0	0	0	0	0	0
	2	3	8		
	↑	0			

1) Negative wt cycle.



(1, 2) (2, 3) (3, 1)